NetSentinel — Explained Like You're New To This

> This document explains the ENTIRE project in **simple words**. The implementation plan stays untouched — this is your **companion guide** to understand it.

🎯 What Is This Project, Really?

Imagine you're a **security guard watching CCTV cameras** at a bank.

- You can **see** everything happening on the cameras
- But you **cannot shout** at the robbers through the screen
- You **cannot lock doors** remotely
- You can ONLY **watch and report** — "Hey, I see something suspicious at Camera 3!"

That's exactly what this project is, but for internet traffic instead of CCTV.

The Real-World Scenario:

Big organizations (like power plants, military networks, banks) have a special security setup:

```
[Their Real Network] ──→ [One-Way Glass/Diode] ──→ [Monit
                                  ↑
                        Data goes THIS way only.
                        Nothing can go back.
```

The **"One-Way Glass"** is a real hardware device called a **data diode**. It physically makes it impossible to send anything back. This protects the real network — even if a hacker takes over your monitoring room, they can't reach the actual network.

Your Job:

Build an **AI system that sits in the monitoring room** and:

1. **Watches** all the internet traffic flowing through
2. **Detects** 6 types of cyber attacks
3. **Explains** why it thinks something is an attack
4. **Shows** everything on a cool dashboard
5. **Records** every alert on a blockchain (tamper-proof evidence)

🔍 The 6 Attacks You Need To Detect (In Plain English)

Attack 1: DDoS (Distributed Denial of Service)

What it is: Imagine 1 million people all calling the same phone number at the same time. The phone line crashes. That's DDoS.

How to spot it: Suddenly, WAY more traffic than normal from TONS of different IPs.

AI approach: Count how many packets/sec are coming in. If it spikes massively → DDoS alert.

Attack 2: Botnet C2 Beaconing

What it is: A hacker has infected computers with malware. The infected

computers "phone home" to the hacker's server every X minutes to get instructions. This regular "check-in" is called **beaconing**.

How to spot it: One computer connects to the same suspicious server every 2 minutes, like clockwork. Normal humans don't browse that regularly.

AI approach: Look at the timing between connections. If they're suspiciously regular (periodic), it's probably a bot checking in.

Attack 3: DGA Domains & DNS Tunneling

What it is:

- **DGA (Domain Generation Algorithm):** Malware generates random-looking domain names like `xk4jf9a2m.com` to find its hacker's server. Normal websites have readable names like `google.com`.
- **DNS Tunneling:** Hiding secret data inside DNS requests (like hiding a letter inside an envelope that looks like a normal bill).

How to spot it: Random gibberish domain names, unusually long DNS queries, weird record types.

AI approach: Check if the domain name looks random (high entropy = random). Feed domain names into a neural network that learned the difference between `google.com` (normal) and `a8xk2mf9.biz` (DGA).

Attack 4: Malware in Encrypted Traffic

What it is: The traffic is encrypted (HTTPS/TLS) so you CAN'T see what's inside. But malware communicating over HTTPS still has patterns you can spot WITHOUT reading the content.

How to spot it:

- The way the malware says "hello" to its server (TLS handshake)

has a unique fingerprint called **JA4**
- The size and timing of packets follow a pattern (like Morse code — even if you can't read the words, you can recognize the rhythm)

AI approach: Extract the JA4 fingerprint + the first 30 packet sizes and timings. Feed this into a Transformer model (same type of AI as ChatGPT, but for packet patterns instead of words).

Attack 5: Reconnaissance / Port Scanning

What it is: Before an attack, hackers "scan" a network to find open doors (ports). It's like a burglar walking down a street trying every door handle.

How to spot it: One IP tries to connect to 1000 different ports on your server, or connects to 500 different servers. Normal users don't do this.

AI approach: Build a graph (like a social network map) of who is talking to whom. If one node has connections to hundreds of others → scanner detected.

Attack 6: Data Exfiltration

What it is: A hacker is secretly copying/stealing data OUT of the network. Like an employee walking out with USB drives full of company secrets.

How to spot it: Way more data going OUT than coming IN from a specific computer. Normally, you download more than you upload (watching videos, loading web pages). If a computer is uploading 10GB but only downloading 100KB → suspicious.

AI approach: Learn what "normal" traffic looks like. Anything that deviates significantly = potential exfiltration.

🏗️ The 7 Layers — What Each Part Does

Think of the project as a **factory assembly line**:

```
Raw Traffic → Clean It → Extract Clues → AI Analyzes → Ex
   Layer 1      Layer 2      Layer 3        Layer 4
```

Layer 1: INGEST (The Security Camera)

What it does: Captures the raw internet traffic

In simple terms: This is like plugging in the CCTV camera. You point it at the network traffic and start recording.

What you actually build:

- A Python script that reads `.pcap` files (these are recordings of network traffic, like video files but for internet data)
- A traffic simulator that GENERATES fake attack traffic for testing (since you don't have a real network to monitor)
- Integration with **Zeek** (a free tool that reads network traffic and writes organized logs — like a CCTV system that automatically notes "Person entered at 2:30 PM, went to room 4")

Tools to install: Python, Scapy (a Python library for network packets), Zeek

---

Layer 2: STREAMING BUS (The Conveyor Belt)

What it does: Moves data from the camera to the analysis room efficiently

In simple terms: Imagine the CCTV footage needs to go from the camera to 6 different analysts sitting in different rooms. The streaming bus is the conveyor belt / pipe system that delivers the footage to all of them simultaneously without losing anything.

What you actually build:

- Set up **Apache Kafka** (or **Redis Streams** for simplicity) — this is a messaging system that says "here's a new piece of traffic data, everyone who needs it, come get it"
- Configure different "topics" (channels) — one for DNS data, one for TLS data, one for flow data, etc.

Tools to install: Redis (easier) or Apache Kafka (production-grade)

Why you need this: Without it, if your AI model is slow for 1 second, you'd LOSE all the traffic that came during that second. Kafka/Redis holds onto it until everything is processed.

Layer 3: FEATURE ENGINEERING (The Detective's Notepad)

What it does: Looks at raw traffic and writes down important clues

In simple terms: Raw traffic data is like raw CCTV footage — too much information. You need a detective to watch the footage and write notes: "This person visited 47 doors in 3 minutes" or "This person sends a message every exactly 120 seconds."

Those notes are called **features** — they're the clues the AI will use.

What you actually build: 4 Python modules, each extracting different types of clues:

1. **DNS Features** (`dns_features.py`)
    - "How random does this domain name look?" (entropy calculation)
    - "How long is the DNS query?"
    - "How many unique subdomains has this domain generated?"
2. **TLS Features** (`tls_features.py`)

- "What's the JA4 fingerprint of this TLS handshake?"
- "Is the certificate self-signed?"
- "What are the first 30 packet sizes?"

3. **Flow Features** (`flow_features.py`)

    - "How many packets per second from this IP?"
    - "What's the ratio of data going out vs coming in?"
    - "How many different ports is this IP connecting to?"

4. **Temporal Features** (`temporal_features.py`)

    - "Is this connection happening at regular intervals?" (FFT = a math trick to find hidden patterns in timing)
    - "Is this activity happening at an unusual time of day?"

Tools needed: NumPy (math), SciPy (FFT for periodicity), Python

---

Layer 4: AI MODELS (The 6 Expert Analysts)

What it does: Takes the clues from Layer 3 and decides "is this an attack or not?"

In simple terms: You have 6 expert analysts, each specializing in one type of crime:

Expert	Speciality	AI Model Type	Where to Train
Expert 1	DDoS attacks	XGBoost (like a decision tree on steroids)	Kaggle, CPU, 10 min
Expert 2	Botnet beaconing	LSTM (AI that understands time patterns)	Kaggle, GPU, 2 hrs
Expert 3	Fake domains (DGA)	CNN + Transformer (AI that reads text patterns)	Colab, GPU, 3 hrs
Expert 4	Encrypted malware	Transformer (AI that reads packet patterns)	Colab, GPU, 4 hrs

Expert 5	Port scanning	GNN (AI that understands network shape/graphs)	Kaggle, GPU, 2 hrs
Expert 6	Data theft	VAE (AI that learns what "normal" looks like)	Kaggle, GPU, 30 min

Then there's a **7th "boss" model** (Meta-Classifier) that takes all 6 experts' opinions and makes the final call.

What you actually build:

- **Training Notebooks** (Jupyter notebooks on Kaggle/Colab) — you write code to teach each AI model using public datasets
- **Inference Code** (Python files on your local machine) — code that loads the trained models and uses them to analyze new traffic

How training works:

1. Go to Kaggle.com → Create a notebook
2. Load a dataset (e.g., CIC-IDS2017 — a public dataset of labeled network attacks)
3. Write the model code (PyTorch)
4. Click "Run" → Kaggle gives you a free GPU → model trains
5. Download the trained model file (`.onnx` format)
6. Put it in your project folder → your backend loads it for real-time detection

---

Layer 5: INTELLIGENCE (The Report Writer)

What it does: Takes the AI's detection and makes it useful for humans

In simple terms: The AI says "this is 94% likely a C2 beacon." But the analyst asks "WHY do you think that?" This layer answers that question.

Three sub-parts:

1. **XAI (Explainable AI)** — Uses SHAP to say "the top 3 reasons the AI flagged this are: (1) regular timing pattern, (2) JA4 matches known malware, (3) always contacts same server"

2. **MITRE ATT&CK Mapper** — MITRE ATT&CK is a giant catalog of hacking techniques. This part automatically labels each alert: "This is technique T1071 (Application Layer Protocol) under the tactic 'Command and Control'"

3. **Alert Manager** — Packages everything into a standardized JSON format:

```
{
  "alert_id": "...",
  "timestamp": "2026-08-24T12:34:56Z",
  "threat_class": "c2_beaconing",
  "confidence": 0.94,
  "evidence": { ... SHAP values, features ... },
  "mitre": { "tactic": "Command and Control", "techni
}
```

Layer 6: BLOCKCHAIN (The Tamper-Proof Safe)

What it does: Records every alert on a blockchain so no one can delete or change them later

In simple terms: Imagine writing each alert in a notebook with invisible ink that can never be erased. Even better — that notebook is copied to 1000 locations simultaneously. No single person can alter it. That's blockchain.

Why you need this:

- If a hacker breaks in and tries to delete the alerts that caught them → they can't, because it's on the blockchain
- If this evidence goes to court → you can prove it was never tampered with
- The SIH theme is "Blockchain & Cybersecurity" — this is literally the theme requirement!

What you actually build:

1. A **Solidity smart contract** (`AlertRegistry.sol`) — this is a small program that lives on the blockchain. It has one main function: `anchorAlert()` — you give it the hash of an alert, and it permanently records it.

2. A **Python client** (`chain_client.py`) — your backend calls this every time it generates an alert. It hashes the alert JSON and sends the hash to the smart contract.

3. An **IPFS client** (`ipfs_client.py`) — IPFS is like Google Drive but decentralized. You store the full alert evidence here. The blockchain only stores the tiny hash (fingerprint), while IPFS stores the full document.

Tools to install: Node.js, Hardhat (Ethereum development framework), ethers.js

---

Layer 7: DASHBOARD (The Control Room Screen)

What it does: Shows everything on a beautiful real-time web interface

In simple terms: This is the big screen in the security control room. All the CCTV feeds, all the analyst reports, all in one place with pretty charts and maps.

What you actually build: A React web application with 10 panels:

Panel	What It Shows	Think of It As
Live Threat Stream	Alerts scrolling in real-time	A live Twitter feed of attacks
Threat Distribution	Pie chart of attack types	"30% DDoS, 25% C2, ..."
Geo Map	World map with attack locations	Like a weather radar but for attacks
Severity Timeline	Graph of attacks over time	Stock chart showing attack volume

Alert Table	Spreadsheet of all alerts	Excel-like table with filtering
MITRE Heatmap	Grid showing which attacks you detect	Color-coded bingo card
XAI Panel	Why the AI flagged this alert	Bar chart of "reasons"
Knowledge Graph	Connections between attackers, IPs, domains	Social network diagram of hackers
Blockchain Audit	Proof that alerts are on-chain	List with ✓ "verified" badges
System Health	Is the system running OK?	Car dashboard gauges

Tools to install: Node.js, React, Vite, D3.js

👥 What Each Person Actually Does (4 Computers)

Person A (Computer 1) — "The Plumber" 🔧

You handle: Getting data in and making it flow

Your daily job in plain English:

1. Write Python code that reads network traffic files (`.pcap`)
2. Write Python code that creates FAKE attack traffic for testing
3. Write Python functions that calculate clues from the traffic (entropy, packet rates, timing patterns)
4. Set up a FastAPI server (a web server) that serves data to the dashboard
5. Set up WebSocket (live connection) so the dashboard updates instantly

You DON'T need to understand: AI models, React, Blockchain

Person B (Computer 2) — "The Brain"

You handle: Training and running all the AI models

Your daily job in plain English:

1. Go to Kaggle.com and Colab, create notebooks
2. Download public datasets (CIC-IDS2017, CTU-13, DGArchive)
3. Write PyTorch code to build each of the 6 AI models
4. Train them on cloud GPUs (press Run, wait, download the result)
5. Write Python wrapper code that loads the trained models and makes predictions
6. Add SHAP explainability (so the AI can explain WHY it flagged something)
7. Write the MITRE ATT&CK mapping (a simple dictionary: "c2_beaconing" → "T1071")

You DON'T need to understand: Networking, React, Blockchain, Kafka

---

Person C (Computer 3) — "The Artist" 🎨

You handle: The web dashboard (what the user actually sees)

Your daily job in plain English:

1. Create a React + TypeScript project
2. Design a dark, cybersecurity-themed UI (think hacker movie screens)
3. Build 10 visual panels (charts, maps, tables, graphs)
4. Connect to the backend via WebSocket (so alerts appear live)
5. Make it look STUNNING — animations, transitions, glassmorphism effects

You DON'T need to understand: AI models, Networking details, Blockchain

---

Person D (Computer 4) — "The Glue + Chain" ⛓

You handle: Blockchain integration + putting everything together

Your daily job in plain English:

1. Write the Solidity smart contract (AlertRegistry.sol — ~50 lines of code)
2. Deploy it using Hardhat (test blockchain on your laptop)
3. Write Python code that sends alert hashes to the blockchain
4. Set up IPFS for evidence storage
5. Write Docker configuration so the ENTIRE project runs with one command
6. Write documentation (README, architecture docs)
7. Help others debug and integrate their parts
8. Prepare the demo and presentation

You DON'T need to understand: AI model internals, Complex chart rendering

---

📦 How Data Flows — Step by Step

Here's what happens when a packet enters the system:

```
Step 1: A network packet arrives (or we simulate one)
        "192.168.1.5 → 185.234.x.x:443, TCP, 1200 bytes"

Step 2: Zeek parses it into structured logs
        conn.log: {src: 192.168.1.5, dst: 185.234.x.x, po
        ssl.log:  {ja4: "t13d1516h2_8daaf...", sni: "upda

Step 3: Logs go into Kafka/Redis (message queue)
        → Everyone who needs this data gets a copy

Step 4: Feature extractors read from Kafka and calculate
```

```
        dns_features: {entropy: 4.2, subdomain_depth: 5,
        tls_features: {ja4: "t13d...", pkt_sizes: [234, 1
        flow_features: {pps: 150, bytes_ratio: 0.02, fan_
        temporal:      {fft_score: 0.97, beacon_interval:

Step 5: Features sent to all 6 AI detectors simultaneousl
        DDoS detector:     "Not DDoS (confidence: 0.05)"
        C2 detector:       "⚠  BEACON DETECTED (confidenc
        DGA detector:      "Not DGA (confidence: 0.12)"
        Encrypted detector:"Suspicious (confidence: 0.71)
        Recon detector:    "Not scanning (confidence: 0.0
        Exfil detector:    "Normal (confidence: 0.15)"

Step 6: Meta-classifier combines all 6 opinions
        Final verdict: "C2 Beaconing, confidence 0.94, se

Step 7: XAI engine explains WHY
        "Top reasons: periodic timing (0.42), same destin
         JA4 matches Cobalt Strike (0.28)"

Step 8: MITRE mapper adds context
        "Tactic: Command & Control, Technique: T1071"

Step 9: Alert packaged as JSON and sent to:
        → Dashboard (via WebSocket) — analyst sees it ins
        → Blockchain (hash anchored) — tamper-proof recor
        → IPFS (full evidence stored) — permanent storage

Step 10: Dashboard updates
         🚨 New alert appears in Live Threat Stream
         📊 Donut chart updates (C2 count: 15 → 16)
         🗺️  New dot appears on the geo map
         🔗 "On-chain verified ✓" badge shows
```

🛠️ Software You Need To Install (Per Computer)

Everyone Needs:

- **Git** (version control)
- **Python 3.11+** (main programming language)
- **Node.js 18+** (for React dashboard and Hardhat)
- **Docker Desktop** (to run everything in containers)
- **VS Code** (code editor)

Computer 1 (Pipeline) Extra:

- `pip install fastapi uvicorn scapy dpkt numpy scipy redis`
- Zeek (optional — can parse logs without it for demo)

Computer 2 (AI/ML) Extra:

- `pip install torch xgboost scikit-learn shap onnxruntime pandas`
- Kaggle account + Colab account (for GPU training)

Computer 3 (Dashboard) Extra:

- `npm create vite@latest dashboard -- --template react-ts`
- `npm install d3 recharts leaflet socket.io-client cytoscape`

Computer 4 (Blockchain) Extra:

- `npm install --save-dev hardhat @nomicfoundation/hardhat-toolbox`
- `pip install web3 requests` (for Python blockchain client)

🗣️ How to Explain This to Judges (30-Second Version)

> "Critical infrastructure like power plants use **data diodes** — one-way hardware that lets you **see** all network traffic but **never send anything back**.
>
> **NetSentinel** sits behind this diode and uses **6 specialized AI models** to detect DDoS attacks, botnet beaconing, fake domains, malware in encrypted traffic, port scanning, and data theft — all in **real-time, without decrypting anything**.
>
> Every alert is **explainable** (the AI tells you WHY it flagged something), mapped to the **MITRE ATT&CK** framework, and **permanently recorded on a blockchain** so evidence can never be tampered with.
>
> We process **50,000+ flows per second** with under 2-second alert latency."

? Common Questions You Might Have

Q: Where do we get network traffic to test with? A: You use PUBLIC DATASETS (CIC-IDS2017, CTU-13) — they're pre-recorded network traffic with attacks already labeled. You also build a traffic SIMULATOR that generates fake attacks using Scapy.

Q: Do we need a real blockchain (Ethereum mainnet)? A: NO! You run a LOCAL blockchain on your laptop using Hardhat. For the demo, you can also deploy to a FREE testnet (Polygon Mumbai). It works exactly like real Ethereum but costs nothing.

Q: Do we need expensive GPUs? A: NO! You train on Kaggle (free, gives you 2x T4 GPUs, 30 hours/week) or Google Colab (free T4 GPU). The trained models run on normal CPU for inference.

Q: What if we can't finish all 6 detectors? A: Prioritize: DDoS (easiest), C2 Beaconing (most impressive demo), DGA (cool visualization), Encrypted Malware (wow factor). The other 2 can be "designed but not fully trained."

Q: Do we actually need Kafka? A: For the hackathon demo, use **Redis Streams** instead — it's much simpler to set up and does the same job. Kafka is the "production-grade" choice you mention in your architecture docs.

Q: What's ONNX? A: It's a file format for AI models. You train a model in PyTorch → export it as `.onnx` file → load it anywhere with `onnxruntime` (which is very fast and doesn't need PyTorch installed). Think of it as converting a Word document to PDF — it works everywhere.

Q: What's SHAP? A: It's a library that explains AI decisions. You give it a model and an input, and it tells you "feature X contributed +0.42 to the prediction, feature Y contributed -0.12." Like showing your work on a math exam.